

Chapitre 4 :

Introduction à l'apprentissage profond

Contexte de l'apprentissage profond

Jusqu'à présent, nous avons vu que l'**intelligence artificielle** (AI) est à la fois un domaine de la science et de l'ingénierie, et englobe plusieurs champs, parmi lesquels on trouve :

- La représentation des connaissances.
- Les systèmes multi-agents.
- La résolution des problèmes et des jeux.
- Les systèmes experts et la planification.
- Le traitement des langages naturels.
- L'apprentissage automatique.
- La vision par ordinateur.
- La robotique.
- La reconnaissance de formes, etc.

Nous savons également que l'**apprentissage automatique** (ML) fait partie de l'intelligence artificielle et s'applique dans plusieurs de ses domaines.

Un autre domaine, à la fois ancien et nouveau, de l'apprentissage automatique est en évolution accélérée. Il s'agit de l'**apprentissage profond** (DL, *deep learning*).

Présentation succincte de l'apprentissage profond

L'**apprentissage profond** est un sous-ensemble de l'apprentissage automatique, comme l'illustre la figure suivante :

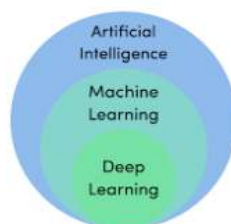


Figure 1. Relation entre AI, ML & DL.

Bref, l'**apprentissage profond** n'est rien de plus que des techniques et des méthodes qui améliorent les algorithmes d'apprentissage automatique, comme par exemple l'**extraction des caractéristiques** (*features extraction*) etc.

L'apprentissage profond est étroitement lié aux **mathématiques**, en particulier l'**optimisation**, la **théorie des graphes**, l'**analyse numérique**, l'**analyse fonctionnelle**, la **théorie des probabilités**, les **statistiques mathématiques**, la **théorie de l'information**, etc.

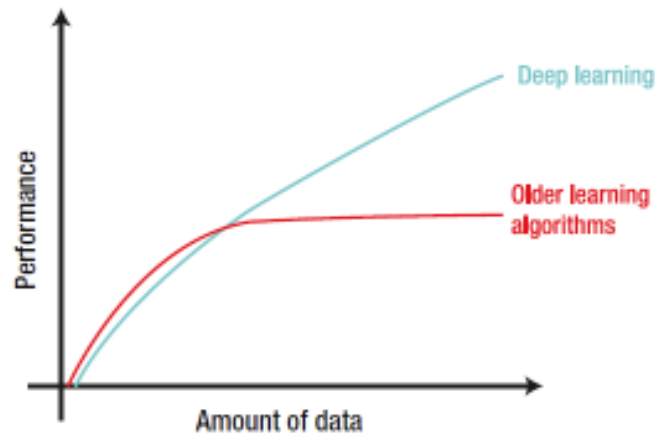
Comme nous l'avons déjà évoqué, il est juste de définir l'apprentissage profond en disant qu'il signifie simplement un apprentissage automatique utilisant des **réseaux de neurones profonds**. C'est vrai que l'apprentissage profond est un sous-ensemble de l'apprentissage automatique, mais il utilise une **représentation plus proche de celle qu'utilise le cerveau humain**.

L'apprentissage profond est fondé sur la manière dont le cerveau humain traite les informations et apprend. Il s'agit d'un modèle d'apprentissage automatique composé de **plusieurs niveaux de représentation**, dans lequel chaque niveau utilise les informations du niveau précédent pour apprendre en profondeur. L'apprentissage profond se compose de réseaux de neurones, les réseaux de neurones se composent de couches et les couches sont constituées d'unités cachées : les **perceptrons**.

Raisons principales qui ont contribué au DL

L'**apprentissage profond** est à la mode de nos jours pour trois raisons principales :

1. Le **big data** :



Les **réseaux de neurones artificiels profonds** s'améliorent avec la présence de grands volumes de données. Nous pouvons se demander si l'énorme quantité de données était également utile pour l'apprentissage automatique. Mais, malheureusement la réponse est négative. En effet, les algorithmes traditionnels de l'apprentissage automatique atteindront certainement un niveau où davantage de données n'améliorera pas leurs performances (c'est le phénomène appelé la **sur-apprentissage**, *overfitting* en anglais).

2. La **puissance de calcul** : les ordinateurs actuels sont de plus en plus puissants et robustes, et nous permet de traiter plus de données en temps raisonnable.
3. Les **algorithmes performants** : le développement d'algorithmes beaucoup plus efficaces grâce au parallélisme, le calcul distribué et aux techniques d'optimisation très poussées, a contribué au relancement de ce champ important de l'apprentissage automatique.

Réseaux de neurones artificiels (RNA)

Nous savons que le **perceptron** est l'unité neuronale la plus simple. Également, nous avons vu que le perceptron est un **classifieur linéaire** (binaire).

Maintenant, nous allons présenter les **réseaux de neurones artificiels (RNA) profonds** et **non profonds**.

La figure suivante illustre la différence entre un RNA profond et un RNA non profond :

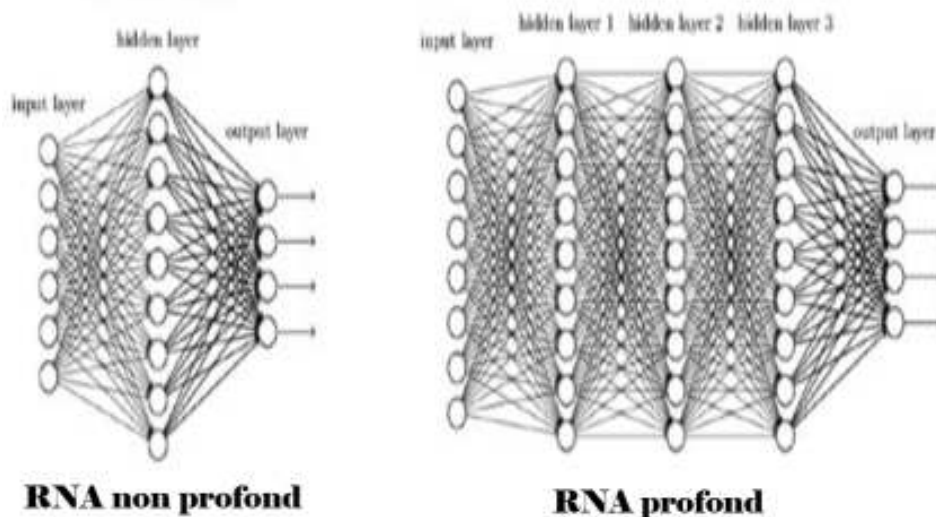


Figure 2. RNA non profond vs. RNA profond.

Un **RNA non profond** se compose :

- D'une **couche d'entrée** (*input layer*) connectée directement aux données d'entrée.
- D'une **couche cachée** (*hidden layer*) de neurones connectés à la couche d'entrée.
- D'une **couche de sortie** (*output layer*) connectée à la couche cachée.

Un **RNA profond** se compose :

- D'une **couche d'entrée** connectée directement aux données d'entrée.
- De **plusieurs couches cachées de neurones**, où chacune est connectée à la suivante. La première couche cachée étant connectée à la couche d'entrée.
- D'une **couche de sortie** (*output layer*) connectée à la dernière couche cachée.

La couche d'entrée

La **couche d'entrée** est formée par un ensemble de **neurones particuliers** dont le rôle est de **capter les données extérieures**. Ces données peuvent par exemple, correspondre à l'amplitude du son d'une musique, aux lettres d'un roman ou d'un livre, ou aux couleurs des pixels d'une image, etc.

Les neurones de la couche d'entrée vont typiquement lire ces données brutes. Un neurone de la couche d'entrée va s'activer si son morceau de

données brutes correspond à son activation. Par exemple, un neurone d'entrée exposé à un pixel très lumineux pourrait beaucoup s'activer.

Les neurones de la couche d'entrée qui ont été activés vont alors envoyer des signaux (**effet de propagation**) le long des **connections** (flèches des réseaux) aux neurones de la première couche cachée, par analogie avec les **synapses** des neurones en biologie.

Formellement, les **données d'entrée** (inputs) sont représentées par un vecteur de valeurs réelles $(x_1, x_2, \dots, x_n)$ de $\mathbb{R}^n$ et les **intensités des connections synaptiques** par un vecteur de poids $(w_1, w_2, \dots, w_n)$. Généralement les poids w_i sont des valeurs réelles comprises entre 0 et 1, c'est-à-dire que $w_i \in [0, 1]$, pour tout indice $i \in \{1, 2, \dots, n\}$. Comme nous l'avons déjà vu, chaque neurone possède une sorte **d'additionneur** qui cumule la somme des produits $x_i \cdot w_i$. Par conséquent, la valeur reçue par un neurone est égale :

$$x_1 \cdot w_1 + x_2 \cdot w_2 + \dots + x_n \cdot w_n$$

À cette somme, le neurone ajoute un biais **b** pour obtenir le niveau d'activation qui est alors égale à :

$$x_1 \cdot w_1 + x_2 \cdot w_2 + \dots + x_n \cdot w_n + b = \mathbf{x} \cdot \mathbf{w} + b$$

La sortie du neurone est obtenue finalement par l'application d'une **fonction d'activation** (non linéaire) appropriée **F** , à cette somme :

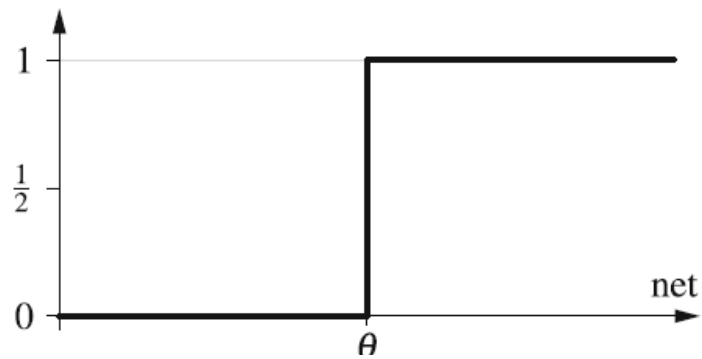
$$\mathbf{y} = F(\mathbf{x} \cdot \mathbf{w} + b)$$

Les fonctions d'activation

Il existe plusieurs fonctions d'activation dans la littérature des réseaux de neurones :

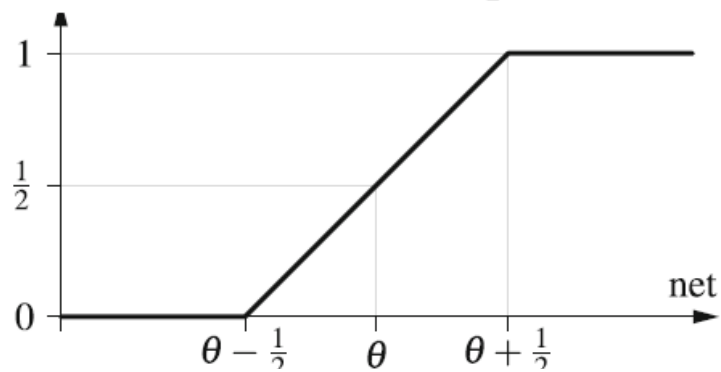
- La fonction de **Heaviside** :

$$f_{\text{act}}(\text{net}, \theta) = \begin{cases} 1 & \text{if } \text{net} \geq \theta, \\ 0 & \text{otherwise.} \end{cases}$$



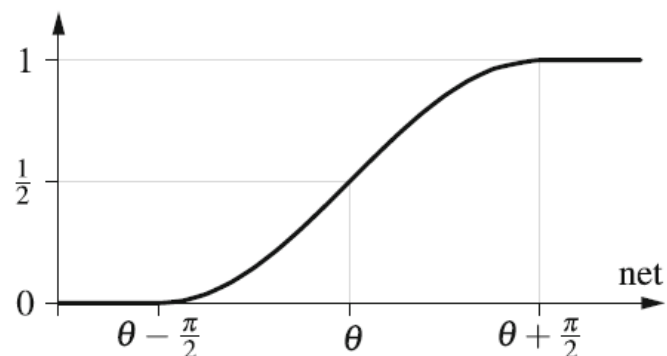
- La fonction **semi-linéaire** :

$$f_{\text{act}}(\text{net}, \theta) = \begin{cases} 1 & \text{if } \text{net} > \theta + \frac{1}{2}, \\ 0 & \text{if } \text{net} < \theta - \frac{1}{2}, \\ (\text{net} - \theta) + \frac{1}{2} & \text{otherwise.} \end{cases}$$



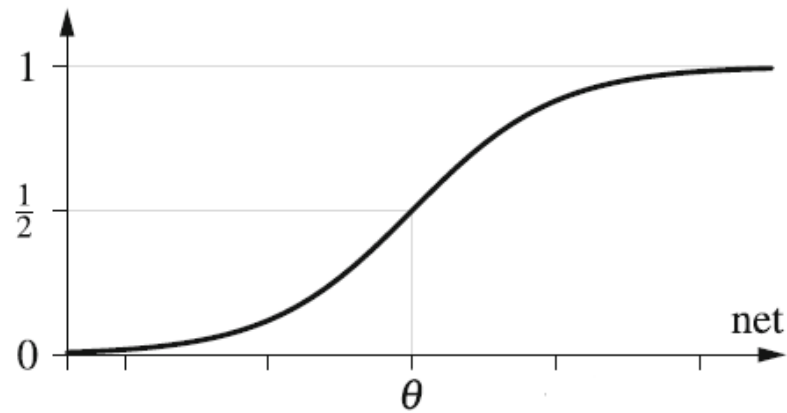
- La fonction **sinus** :

$$f_{\text{act}}(\text{net}, \theta) = \begin{cases} 1 & \text{if } \text{net} > \theta + \frac{\pi}{2}, \\ 0 & \text{if } \text{net} < \theta - \frac{\pi}{2}, \\ \frac{\sin(\text{net} - \theta) + 1}{2} & \text{otherwise.} \end{cases}$$



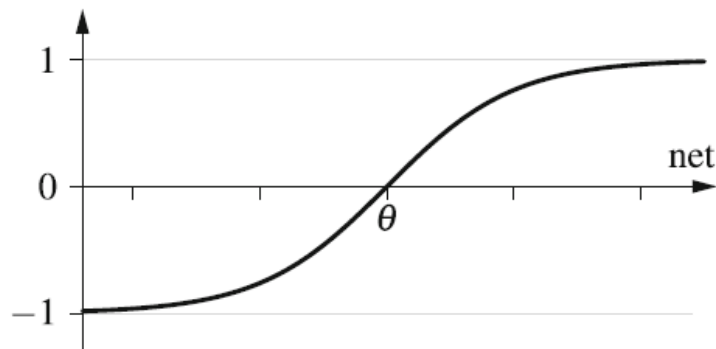
- La fonction **sigmoïde** :

$$f_{\text{act}}(\text{net}, \theta) = \frac{1}{1 + e^{-(\text{net} - \theta)}}$$



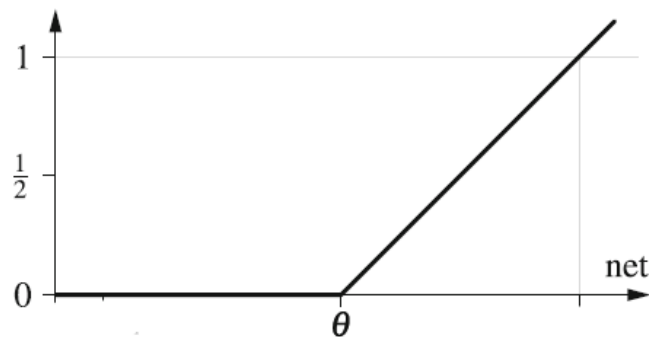
- La fonction **tangente hyperbolique** :

$$f_{\text{act}}(\text{net}, \theta) = \tanh(\text{net} - \theta) = \frac{2}{1 + e^{-2(\text{net} - \theta)}} - 1$$

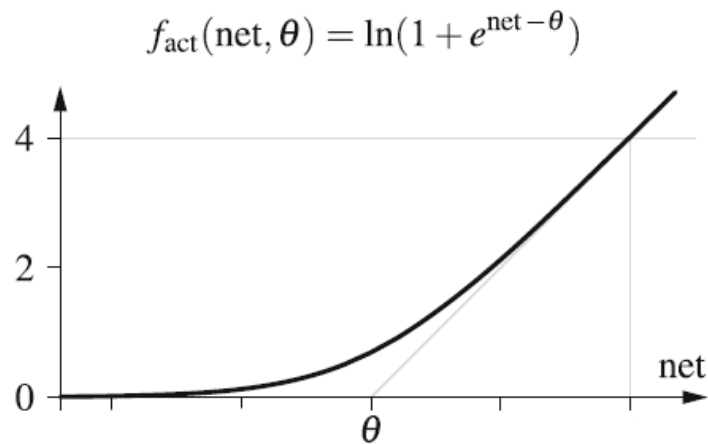


- La fonction **ReLU** (*Rectified linear unit*) :

$$f_{\text{act}}(\text{net}, \theta) = \max\{0, \text{net} - \theta\}$$



- La fonction **SoftPlus** :



- La fonction **SoftMax** :

$$f_i(\vec{a}) = \frac{e^{a_i}}{\sum_k e^{a_k}}$$

Remarque : toutes les fonctions précédentes sont **différenciables**, ce qui permet rend facile l'apprentissage du réseau avec la méthode de la **descente de gradient**.

Les couches cachées

Une **couche cachée** est également formée par un ensemble de **neurones** dont le rôle est de **recevoir/envoyer** des signaux. C'est là que les choses intéressantes commencent. En particulier ce qui va changer complètement le comportement du réseau de neurones, c'est la manière dont un neurone de cette couche va s'activer ou non en fonction des données qu'on lui envoie.

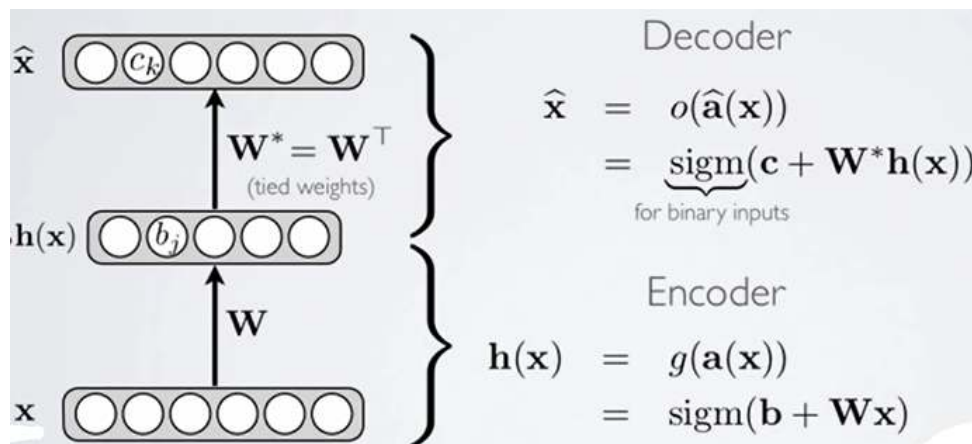
Réseaux de neurones FeedForward : FFNN

Ce sont les types de réseaux neuronaux les plus couramment utilisés. Ces réseaux calculent une série de transformations, d'une façon **unidirectionnelle**, entre leur entrée et leur sortie aboutissant à une nouvelle représentation de l'entrée à chaque couche successive.

Les FFNNs sont des **classificateurs** multicouches comportant de nombreuses couches de poids séparées par des fonctions non-linéaires (**sigmoïde**, **tanh**, **ReLU**, **selu**, **softmax**). Ils sont également connus sous le nom de **Perceptrons multicouches** (MLP).

Mais on peut également avoir des applications plus subtiles comme par

exemple **l'extraction de sémantiques** à partir des données brutes. Dans ce cas, on peut avoir généralement plusieurs neurones finaux dont les activations seront combinées pour déduire une sémantique spécifique. Mathématiquement, on peut bêtement collecter tous les niveaux d'activation pour former un vecteur, ce qui nous donne une représentation vectorielle des données brutes. Avec cette technique, les FFNN sont utilisés pour **l'apprentissage non supervisé** des attributs en tant qu'**auto-encodeurs**.



MLP : MultiLayer Perceptrons

Les **perceptrons multicouches** (MLP) appartiennent à la catégorie des FFNNs et sont constitués de trois couches : une couche d'entrée, une ou plusieurs couches cachées et une couche de sortie finale. Les MLP sont caractérisés par le fait qu'ils utilisent des **fonctions d'activation linéaires** pour la couche d'entrée.

Exemple d'un perceptron simple en python

Nous allons implémenter un perceptron simple avec la bibliothèque **sklearn** de python pour la fonction booléenne AND.

```
from sklearn.linear_model import Perceptron

# simple perceptron pour la fonction logique AND

X = [[0, 0], [0, 1], [1, 0], [1, 1]] # input layer
y = [0, 0, 0, 1]                     # output layer

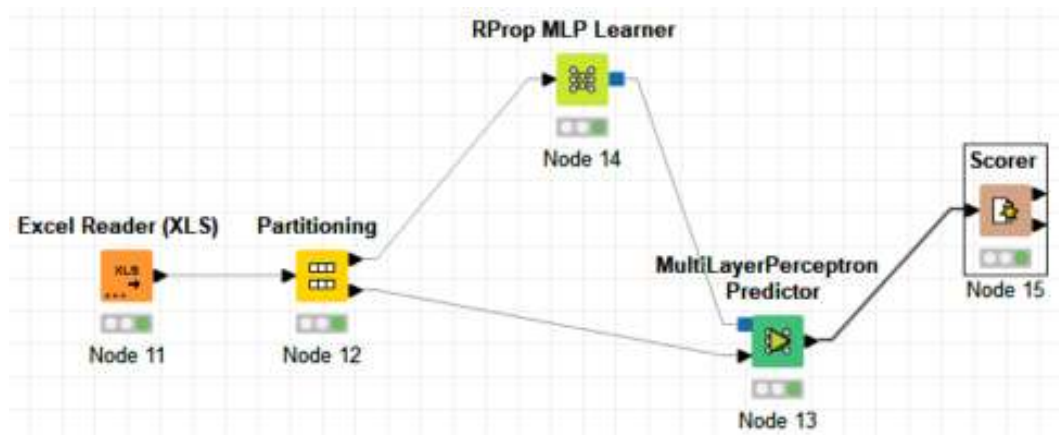
my_pcrt = Perceptron()
my_pcrt.fit(X, y)
```

```
X0 = [1, 0]
```

```
print('Class of input', X0, '==>', my_pcert.predict([X0]))  
print('Accuracy:', my_pcert.score(X, y))
```

Exemple d'un MLP en Knime

Maintenant, nous allons utiliser Knime pour la classification des plantes en utilisant le dataset Iris. Cette classification est réalisée au moyen d'un réseau de type FFNN (MLP). Voici son schéma :



La rétropropagation

La **rétropropagation** (de l'erreur) est une technique avec laquelle les chercheurs en ML ont pu résoudre le problème du XOR posé par Minsky et son équipe. Son objectif est de **minimiser l'erreur** entre la classe désirée (réelle) et la classe obtenue par le réseau de neurones (prédite).

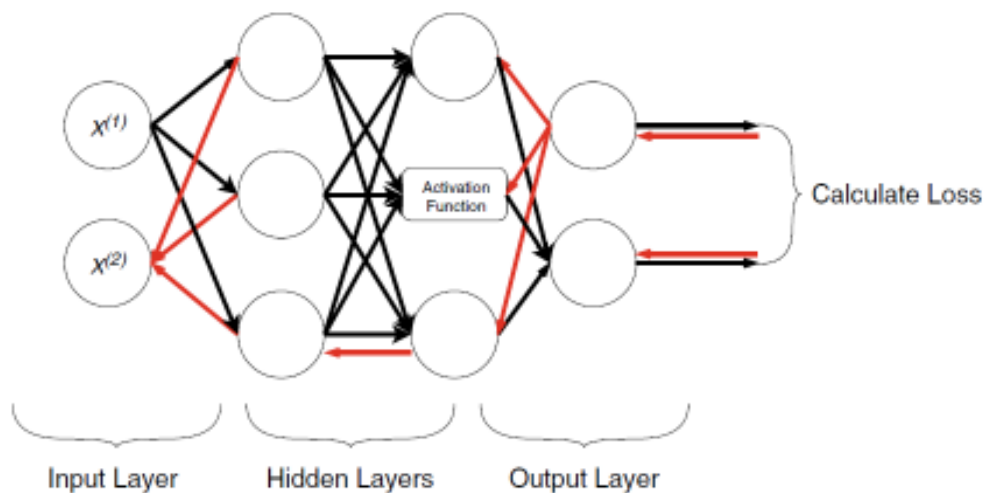


Figure 3. Un RNA utilisant la rétropropagation

Cette technique utilise l'algorithme suivant, appelé algorithme de rétropropagation :

Algorithm 1: Forward and Backward propagation in CNN

```

Input:  $x = [x_1, \dots, x_N]^T$ 
begin
  for  $l := 1 \rightarrow \#HiddenLayers$  do
    for  $i := 1 \rightarrow \#RowunitinLayerl$  do
      for  $j := 1 \rightarrow \#ColumnunitinLayerl$  do
        Find the layer activations by,  $y_{ij}^l = \varphi(x_{ij}^l + b_{ij}^l)$ 
        Compute next layer inputs
      end
    end
  end
  Keep the final output as  $y^l$ 
  Calculate error at the output layer.
  begin
    for  $l := \#HiddenLayers \rightarrow 1$  do
      Find error partial derivation by  $\frac{\partial E}{\partial x_{ij}^l} = \frac{\partial E}{\partial y_{ij}^l} \frac{\partial y_{ij}^l}{\partial x_{ij}^l} = \frac{\partial E}{\partial y_{ij}^l} \frac{\partial}{\partial x_{ij}^l} (\psi(x_{ij}^l))$ 
      Find error at the previous layer.
    end
    Calculate the gradient of the error by  $\partial \omega_{ab} = \sum_{i=0}^{N-n} \sum_{i=0}^{N-n} \frac{\partial E}{\partial x_{ij}^l} \frac{\partial x_{ij}^l}{\partial \omega_{ab}} = \sum_{i=0}^{N-n} \sum_{i=0}^{N-n} \frac{\partial E}{\partial x_{ij}^l} y_{(i+a)(j+b)}^{l-1}$ 
  
```

Figure 4. L'algorithme d'apprentissage avec rétropropagation

La première phase consiste en une **passé en avant** (*forward pass*) pour **calculer les poids** de chaque couche en fonction des poids de la couche qui la précède. Le calcul dans cette étape se résume par l'équation suivante :

$$A^l = f(W^l A^{l-1} + b^l)$$

Après la passe en avant, l'algorithme calcul l'erreur (moyenne des carrés des écarts) en utilisant la fonction suivante :

$$C = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

La deuxième phase consiste en une **passé en arrière** (*backward pass*) pour **ajuster les poids** de chaque couche en commençant par la dernière couche jusqu'à la première. L'ajustement des poids est réalisé en utilisant la formule de la **descente du gradient** :

$$\Delta w = -\eta \frac{\partial C}{\partial w}$$

Types de réseaux de neurones artificiels profonds

Les **RNA profonds** existent en plusieurs types, parmi lesquels on peut citer :

- Réseaux de neurones à convolution (CNN).
- Réseaux de neurones récurrents (RNN).
- Auto-encodeurs profonds (DAE).
- Réseaux de croyances profonds (DBN).
- Réseaux d'adversaires génératifs (GAN).
- Réseaux de neurones récurrents.

Réseaux de neurones à convolution : CNN

Les réseaux de neurones à convolution CNNs (**CNN** : *Convolutional Neural Network*) tentent à simuler les opérations effectuées par l'œil humain lors du processus de la vision. Un CNN est un type particulier de RNA qui se base sur l'opérateur de **produit de convolution**. Les CNN sont dérivés des architectures de type MLP, cependant ils utilisent des **poids partagés**, liés à la fenêtre de convolution, qui leur permettent une **extraction implicite de caractéristiques locales**.

Les biologistes ont découvert que le **cortex visuel** de nombreuses espèces animales, et en particulier les mammifères, avait une **structure distinctive**, celui-ci se décompose en 5 régions successives appelées originalement V_1 , V_2 , V_3 , V_4 et V_5 .

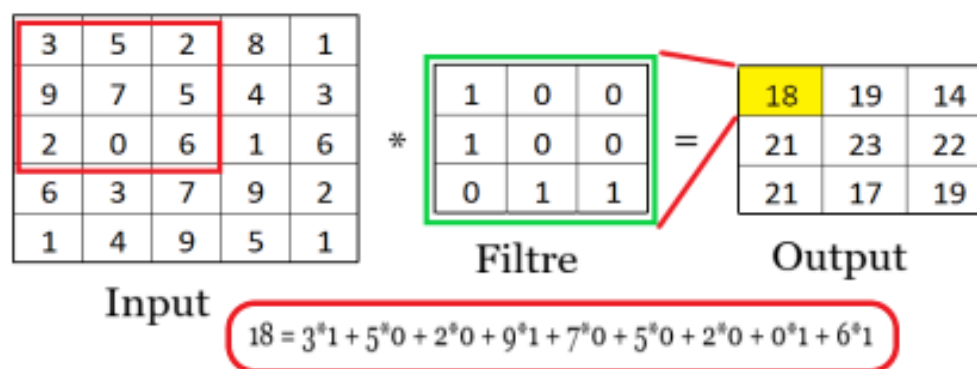
Le chercheur en deep learning **Yann Le Cun** et ses collaborateurs ont proposé d'effectuer une analyse d'images qui passait par un certain nombre de couches de neurones. Intuitivement, chaque couche correspond au calcul de **représentation** de plus en plus abstraite du contenu de l'image.

Typiquement les neurones de la couche d'entrée mesurent uniquement la luminosité et la couleur des pixels. La couche qui suit va calculer différentes sortes de corrélation entre des pixels voisins, puis la couche suivante va mettre en bout ces corrélations pour identifier des lignes

directrices, puis la couche suivante regroupera ces lignes directrices pour identifier des structures, puis on essaiera d'identifier des composants comme des yeux ou des oreilles, puis on combinera la présence d'yeux et d'oreilles pour inférer des concepts plus abstraits comme la présence ou l'absence d'un chat. Le réseau de neurones calculera ainsi une **abstraction croissante des données** qui lui sont représentées, ce qui lui permet de calculer en bout de ligne une **représentation sémantique de l'image** avec toutes les applications dont on a déjà parlé dans les séances précédentes.

Ce principe général de superposition de couches de réseaux de neurones n'est pas du tout spécifique aux **réseaux de neurones à convolution (CNN : Convolutional Neural Network)**. L'idée importante des CNNs c'est surtout que chaque neurone d'une couche intermédiaire n'est exposé qu'à un **champ récepteur particulier** ainsi que le fait que l'analyse de ce champ récepteur est la même que l'analyse qu'effectue un autre neurone de la même couche avec son propre champ récepteur.

Pour illustrer cela, prenons le cas d'un neurone de la première couche cachée. Ce neurone va typiquement analyser une toute petite région du champ visuel. Prenons le cas simpliste où cette région ne contient que 9 pixels (3x3) avec différents niveaux de luminosité. Le neurone va alors multiplier la luminosité de chaque pixel par un poids synaptique (éventuellement négatif) et calculer la somme des luminosités pondérées, et puis ensuite - ce qui est souvent fait - c'est de ne retenir que la partie positive du résultat, ce qui correspond à la fonction d'activation **ReLU**.



Les poids synaptiques forment alors une matrice appelée **matrice de convolution** (on parle aussi parfois de **filtre** ou **Kernel**). L'idée générale des CNNs est d'utiliser la même matrice de convolution pour tous les champs récepteurs de l'image. De plus, les CNNs utilisent les mêmes poids synaptiques, on dit alors qu'il y a **partage de poids synaptiques**.

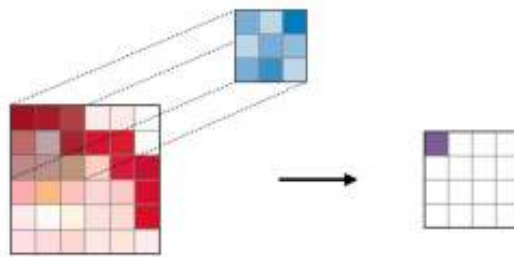
Donc, comme conclusion, un RNA est dit **convolutif**, lorsque chaque neurone reçoit ses informations non pas de toute la couche précédente, mais seulement des neurones situés dans son champ réceptif.

Les CNNs sont plus adaptés au **traitement d'images** et au **traitement des langages naturels**.

La couche de convolution

Quand on lui présente une nouvelle image, le CNN ne sait pas exactement si les caractéristiques seront présentes dans l'image ou où elles pourraient être, il cherche donc à les trouver dans toute l'image et dans n'importe quelle position. En calculant dans toute l'image si une caractéristique est présente, nous faisons un **filtrage**. Les mathématiques que nous utilisons pour réaliser cette opération sont appelés une **convolution**, de laquelle les réseaux de neurones à convolution tiennent leur nom. La couche de convolution est la composante clé des réseaux des CNN, et constitue toujours au moins leur première couche. Son but est de repérer la présence d'un ensemble de caractéristiques (*features*) dans les images reçues en entrée.

Elle fonctionne comme un **extracteur de caractéristiques des images**. Une image est passée à travers une succession de filtres, ou noyaux de convolution. On obtient alors pour chaque paire (image, filtre) une carte de caractéristiques (*feature map*), qui nous indique où se situent les caractéristiques dans l'image : plus la valeur est élevée, plus l'endroit correspondant dans l'image ressemble à la caractéristique. Finalement, les valeurs des dernières cartes d'activation sont concaténées dans un vecteur. Ce vecteur définit la sortie du premier bloc, et l'entrée du second.



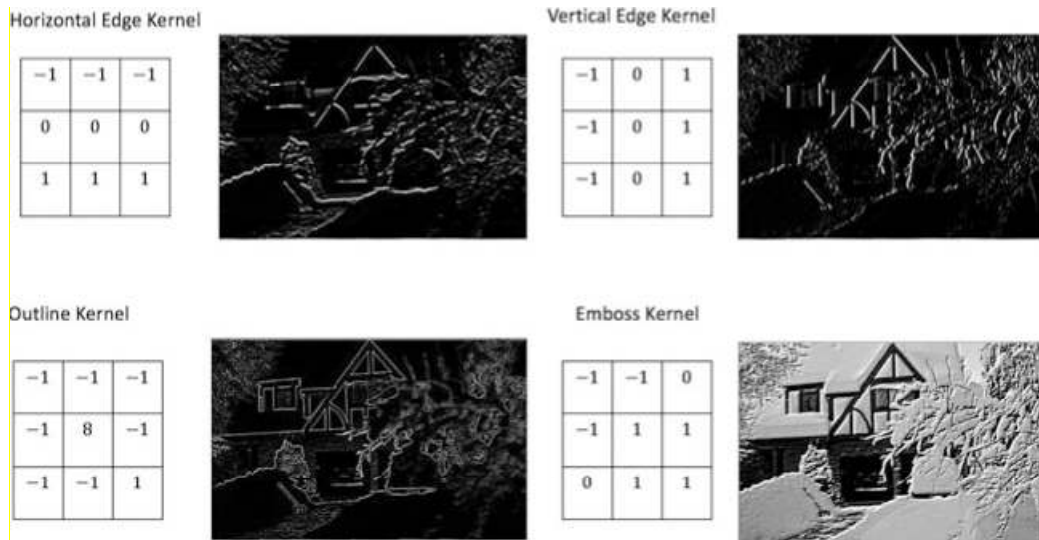


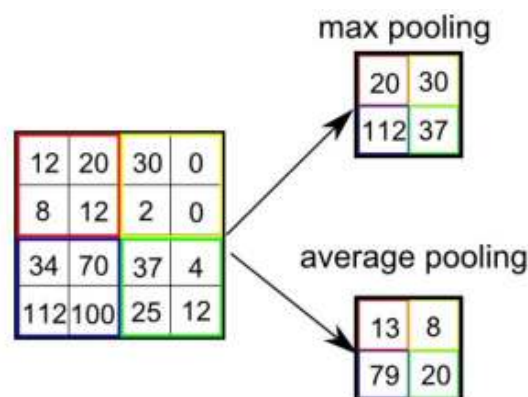
Figure : Quelques exemples de filtres classiques

La couche de pooling

Un autre outil très puissant utilisé par les CNNs est le **Pooling**. C'est une méthode qui permet de prendre une large image et d'en réduire la taille tout en préservant les informations les plus importantes qu'elle contient. C'est donc une technique de **réduction de la dimensionnalité**.

Le **pooling** est une fonction de **sous-échantillonnage spatial**. Elle prend toujours en entrée des feature maps de taille $N_x \times N_y \times D$, et réduit les deux premières dimensions spatiales. Les deux méthodes les plus utilisées pour appliquer cette opération sont les suivantes :

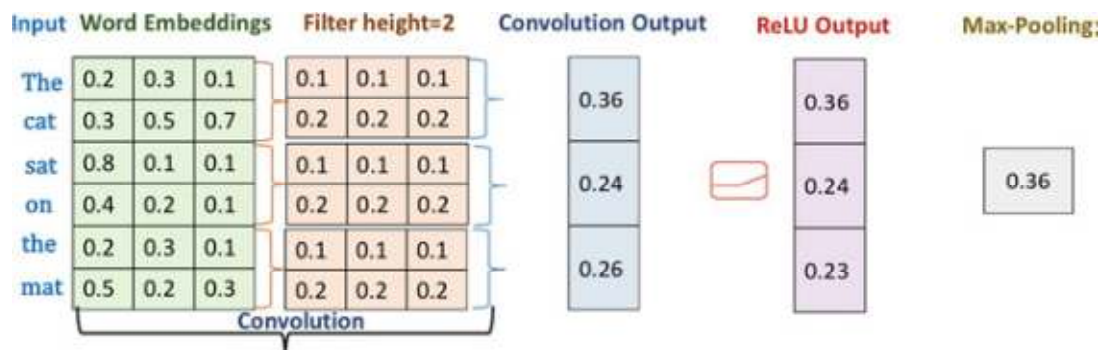
- Le **pooling average** : on prend la moyenne des valeurs de chaque zone.
- Le **pooling max** : on extrait uniquement la valeur la plus élevée de chaque zone.



Il y en a encore d'autres : **L2-Norm**, **Stochastic** et **Spectral pooling**.

La technique du **pooling** permet alors **d'améliorer l'efficacité du réseau** et éviter le **sur-apprentissage**.

La figure suivante illustre un fragment d'un réseau CNN dans le cadre du NLP :



La fonction d'activation pour un CNN

Pour les CNN, une fois la convolution effectuée, une fonction d'activation est appliquée sur toutes les valeurs de l'image filtrée. Les fonctions d'activations les plus utilisées sont : **sigmoïde**, **ReLU** et **SoftMax**.

Les couches entièrement connectées

Les couches CNNs précédemment décrites ciblent essentiellement certaines parties spatiales de l'entrée et les transforment à l'aide des **filtres convolutifs**.

Les **couches entièrement connectées** rassemblent ces informations de manière traditionnelle comme dans un MLP pour générer la sortie souhaitée. Elles utilisent généralement la fonction d'activation **softmax** comme étape supplémentaire pour normaliser la sortie. Supposons que l'entrée du softmax soit un vecteur y de dimension n .

La fonction Softmax est définie par la formule suivante :

$$\sigma(y)_j = \frac{e^{y_j}}{\sum_{i=1}^n e^{y_i}}$$

La fonction **Softmax normalise** la sortie pour qu'elle s'additionne à 1.

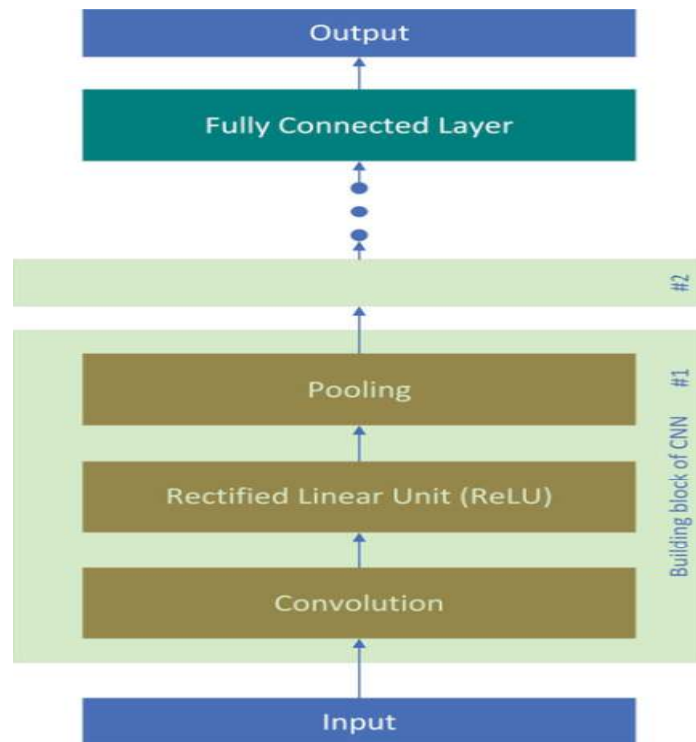


Figure : architecture générale d'un réseau CNN

Apprentissage d'un réseau CNN

Le processus d'apprentissage dans les réseaux **CNN** s'effectue généralement en utilisant un algorithme de **descente de gradient stochastique**. Voici les principales étapes :

1. **Collecter** les données nécessaires pour l'apprentissage et les annoter.
2. **Initialiser** les **poids synaptiques** et les **coefficients du filtre de convolution**.
3. **Sélectionner** un échantillon aléatoire unique (ou un mini-lot d'échantillons), puis l'entraîner à travers le réseau. Générer le résultat (étiquette de classe en cas de classification ou de valeur réelle en cas de régression).
4. **Comparer** la **sortie du réseau** avec la **sortie attendue**, puis utiliser la méthode de la **rétropropagation d'erreur** pour mettre à jour les coefficients de filtre et les poids synaptiques à chaque couche.
5. **Répéter** les étapes 3 et 4 jusqu'à ce que l'algorithme converge vers les niveaux d'erreur souhaités.

6. Afin d'ajuster les **hyperparamètres** (taille des noyaux de convolution, nombre de blocs de convolution, etc.), répéter le processus d'apprentissage entier plusieurs fois pour différents jeux d'hyperparamètres, puis en choisir ceux qui fonctionnent le mieux.

Réseaux de neurones récurrents : RNN

Tous les réseaux de neurones traditionnels vus jusqu'à présent produisent des **modèles d'apprentissage statiques**. Ils travaillent avec des données déjà collectées et qui ne changent au cours du temps.

Les **réseaux de neurones récurrents** ou **RNN** proposent un cadre pour traiter des **données dynamiques** ou **séquentielles** qui changent avec le temps. Les RNN présentent un concept **d'état** qui est fonction du temps. Les RNN classiques, parfois appelés réseaux entièrement récurrents, ont une architecture très similaire à un MLP, mais avec l'ajout d'une **rétroaction de l'état actuel**, comme le montre la figure ci-dessous :

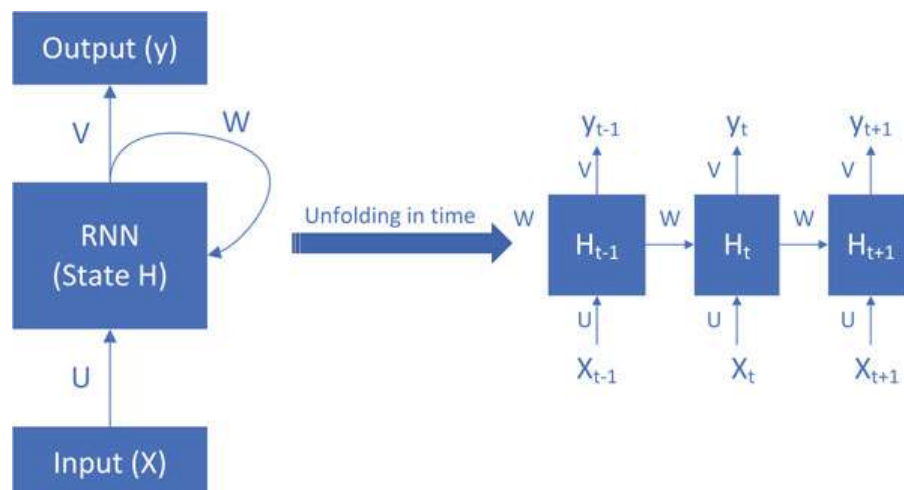


Figure : schéma général d'un RNN

Applications des RNN

Les RNN s'utilisent surtout dans les applications suivantes :

- La reconnaissance de la parole.
- La traduction automatique des langues naturelles.
- La composition musicale.
- La reconnaissance de l'écriture manuscrite.
- L'apprentissage des règles grammaticales d'une langue.

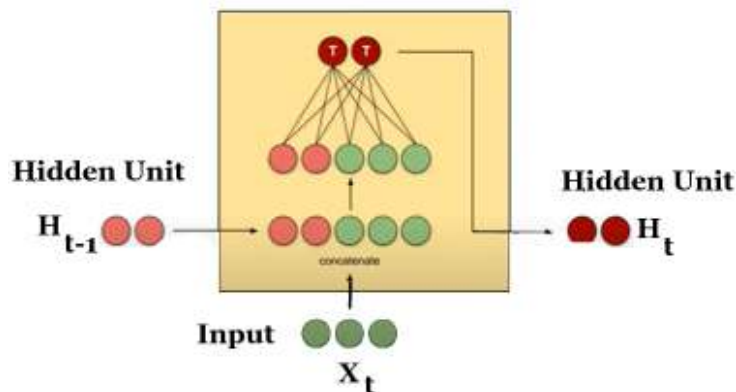
- La prédiction des stocks.
- La conduction automatique des voitures autonomes.

Équations d'un RNN

D'une manière générale, un RNN se compose d'une couche d'entrée, d'un certain nombre de couches cachées et d'une couche de sortie. Cependant, certains neurones de la couche cachée sont dotés d'**états**. À chaque instant t , l'état du neurone de ce type change. L'état d'un RNN est mis à jour à chaque instance de temps en utilisant l'entrée et l'état précédent comme suit :

$$H_t = f_H(U \cdot X_t + W \cdot H_{t-1})$$

La figure suivante illustre bien cette équation de transformation d'état :



Les RNN disposent alors d'une **boucle de répétition** autour de l'état. Ainsi, la sortie finale du RNN peut être donnée comme suit :

$$y_t = f_y(V \cdot H_t)$$

Cette caractéristique de la présente d'une boucle sur l'état d'un RNN lui donne la capacité d'utiliser les **opérations de mémoire**. Donc, avec un RNN on peut prédire des données à un instant t à partir des données à l'instant $(t - 1)$.

Limites des RNN

Les RNN ont réussi à modéliser des données de **séries chronologiques** qu'il n'était pas possible de modéliser à l'aide de réseaux de neurones traditionnels qui traitaient des données statiques. Cependant, lorsque la séquence de données à analyser est longue, les RNN s'avèrent inefficaces à s'adapter à ces conditions en raison des problèmes d'explosion et de disparition des gradients et/ou des poids oscillants.

Les algorithmes d'apprentissage avec un RNN semblaient atteindre une limite après qu'un certain nombre d'échantillons aient été consommés et toutes les mises à jour après ce point étaient minuscules et ne changeaient pas vraiment le comportement du réseau. Dans certains cas, en raison de la forte volatilité des changements dans les échantillons d'apprentissage, les poids du réseau continuaient à osciller et ne convergeraient pas.

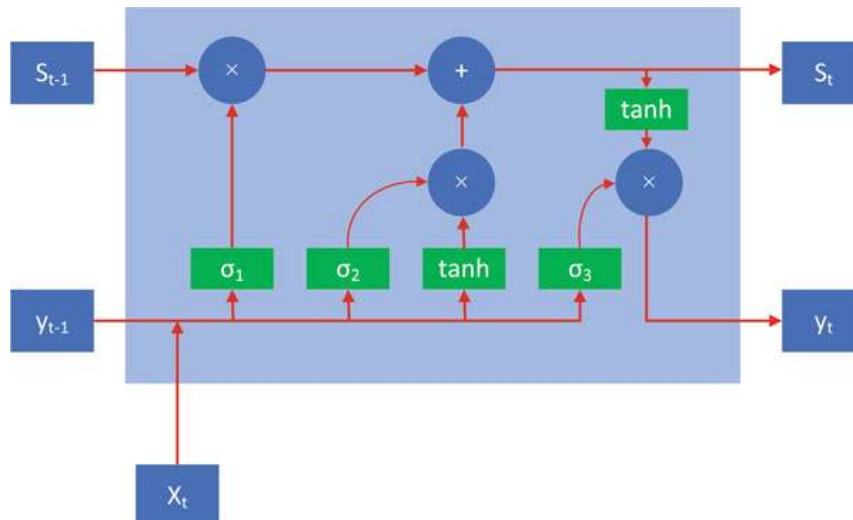
Apprentissage dans les réseaux CNN

Compte tenu de la représentation dépliée des RNN, on comprend que le réseau puisse être appris à travers de nombreuses étapes de temps dans l'algorithme de rétropropagation. Cet algorithme est appelé **rétropropagation dans le temps (BPTT)**. Le BPTT a mérité ce nom, car il aura des signaux d'erreur refluant des pas de temps futurs ainsi que de la couche supérieure. L'apprentissage de RNN avec BPTT est très coûteux en temps de calcul. De plus, deux phénomènes bien connus dans l'apprentissage d'un réseau de neurones profonds et qui sont liés au gradient (disparition ou explosion du gradient) peuvent se produire lors de la rétropropagation d'erreur sur de nombreuses couches. Une variante de BPTT appelée **Truncated BPTT (TBPTT)** qui limite le nombre de pas de temps où l'erreur peut être propagée, peut mieux gérer certains des problèmes ci-dessus. Cependant, en limitant la propagation des erreurs, TBPTT peut finir par réduire la longueur de la dépendance apprise.

LSTM-RNN : Long Short-Time Memory RNN

Hochreiter et **Schmidhuber** ont proposé une amélioration de l'architecture RNN standard sous la forme de **mémoire de longue et courte durée** RNN ou **LSTM-RNN**. Cette architecture a amélioré les RNN pour surmonter la plupart de leurs limitations.

Le changement fondamental apporté par cette architecture est l'utilisation de la **porte de l'oubli (forget gate)**. Le LSTM-RNN avait tous les avantages du RNN avec des limitations bien réduites. En conséquence, la plupart des applications RNN modernes sont basées sur l'architecture LSTM. La figure suivante montre les composants du LSTM-RNN.



L'architecture est assez complexe avec plusieurs opérations de portes. Dans la figure ci-dessus :

- S_t désigne l'état du LSTM à l'instant t .
- y_t désigne la sortie du LSTM au temps t .
- x_t désigne le vecteur d'entrée à l'instant t .
- σ_1 , σ_2 et σ_3 désignent respectivement la porte d'oubli, la porte d'entrée et la porte de sortie.

Regardons les opérations à chaque porte.

La porte d'oubli : forget gate

À la porte d'oubli, l'entrée x_t est combinée avec la sortie précédente y_{t-1} pour générer une fraction entre 0 et 1, qui détermine combien l'état précédent doit être préservé (ou en d'autres termes, quelle part de l'état doit être oublié). Cette sortie est ensuite multipliée par l'état précédent S_{t-1} .

La porte d'entrée

La porte d'entrée fonctionne sur les mêmes signaux que la porte d'oubli - c'est-à-dire x_t et y_{t-1} - mais ici l'objectif est de décider quelles nouvelles informations vont entrer dans l'état de LSTM. La sortie de la porte d'entrée (à nouveau une fraction entre 0 et 1) est multipliée par la sortie du bloc « $\tanh$ » qui produit les nouvelles valeurs qui doivent être ajoutées à l'état précédent S_{t-1} . Ce vecteur est ensuite ajouté à l'état précédent pour générer l'état actuel S_t .

La porte de sortie

À la porte de sortie, l'entrée et l'état précédent sont déclenchés comme précédemment pour générer une autre fraction de mise à l'échelle qui est combinée ensuite avec la sortie du bloc « **tanh** » qui apporte l'état actuel **St**. Cette sortie est ensuite retournée. La sortie et l'état sont renvoyés dans le bloc LSTM comme indiqué.

Domaines d'application de l'apprentissage profond

Les domaines d'application du DL sont très variés. Nous citons alors les plus courants de ces domaines :

- **Vision par ordinateur et traitement d'images** : les voitures autonomes, la classification d'image, la segmentation d'image, la détection / la localisation / le suivi d'objet, la reconnaissance de visage et d'action humaines, la reconstruction d'image, la synthèse d'image, l'amélioration d'image, le sous-titrage d'image, etc. En raison de leur capacité unique d'apprentissage des caractéristiques, la majorité des méthodes utilisent les réseaux CNN sur différentes applications de vision par ordinateur. Parmi les autres modèles DL utilisés en vision par ordinateur, il y a aussi l'auto-encodage profond, les réseaux de croyances profondes, les machines de Boltzmann profondes et les GAN.
- **Traitement du langage naturel** : la reconnaissance d'entités nommées (NER), la classification de phrases, l'analyse des sentiments, la correspondance requêtes-documents, la correspondance sémantique, la génération de langage naturel, la modélisation de langage, la traduction automatique, le résumé automatique de textes, etc. En raison de leur capacité à traiter des séquences, les réseaux RNN et leurs variantes sont largement utilisés dans les diverses applications TALN. Les réseaux CNN et leurs variantes ont été utilisés pour des tâches TALN simples telles que la reconnaissance d'entités nommées (NER) et des tâches plus complexes impliquant des longueurs variables de textes. Des réseaux de neurones récurrents ont été également utilisés pour l'analyse, la classification des relations sémantiques, et la relation entre les phrases.
- **Traitement de la parole** : la reconnaissance téléphonique, la reconnaissance vocale, la synthèse vocale, le traitement du signal musical, la récupération d'informations musicales, l'analyse

musicale, l'estimation de la similitude musicale, la surveillance acoustique, la localisation et le suivi des sources sonores, la séparation des sources, l'amélioration audio. Différents types de modèles d'apprentissage profond tels que les DNN, CNN, RNN, RBM, DBN, GAN, VAE et leurs hybrides ont été appliqués dans ces applications.

- **Biologie** : la prédiction de cibles moléculaires, la prédiction de structures protéiques, la prédiction de contacts résidu-résidu protéiques, la prédiction d'éléments génomiques fonctionnels (les promoteurs), l'analyse de données d'expression génique, etc. Parmi les modèles DL les plus fréquemment utilisés dans ces applications, il y a les réseaux CNN, RNN profond et les auto-encodeurs de dé-bruitage.
- **Santé et médecine** : la classification d'images médicales / biologiques, la découverte de médicaments, la prédiction de nouvelles cibles de médicaments, le sous-typage de maladies, le diagnostic précoce de maladies, la catégorisation des patients, la recherche d'informations, l'analyse des dossiers de santé électroniques, etc. Les CNN, auto-encodeurs profonds, DBN et RNN sont les modèles les plus couramment utilisés dans ces applications.
- **Applications métiers** : l'analyse boursière, la détection de fraude financière, la publicité mobile, la gestion de la relation client, le marketing social, les systèmes de recommandation, la détection d'anomalies, la prédiction des risques, la cybersécurité. La plupart de ces applications utilisent CNN et RNN.

En outre, on trouve les applications de l'apprentissage profond dans l'ingénierie de contrôle, la robotique, la fabrication intelligente, les jeux automatiques, la mise en réseau, la cybersécurité, l'analyse de données volumineuses, le traitement des arts visuels, l'armée et la défense.

Plateformes de l'apprentissage profond

Il existe de nombreuses plates-formes d'apprentissage profond :

- **Caffe** : prend en charge la détection et la classification d'objets visuels ainsi que la segmentation d'images à l'aide des techniques CNN, R-CNN, LSTM et de réseaux de neurones entièrement connectés. Caffe prend en charge l'accélération basée sur GPU et

CPU. Caffe2 inclut de nouvelles fonctionnalités telles que les réseaux de neurones récurrents. Fin mars 2018, Caffe2 a été fusionné avec PyTorch.

- **TensorFlow** : **plateforme** de deep learning de Google.
- **PyTorch** : principalement développé par le "AI Research Lab" de Facebook. C'est une **bibliothèque** de ML open source pour des applications telles que la vision par ordinateur et le TALN.
- **MXNet** : a été adopté par Amazon Web Services qui prend en charge de nombreux langages de programmation (C++, Python, R, Julia, etc). C'est un **framework** flexible et ultra-scalable de DL contenant la technologie de pointe en matière d'apprentissage profond (CNN, LSTM).
- **Theano** : c'est un compilateur pour les expressions mathématiques symbolique développé par MILA (*Montreal Institute for Learning Algorithms*). C'est l'une des bibliothèques les plus stables, qui permet des calculs automatiques de gradient de fonction avec son interface Python.
- **Python** : avec ses bibliothèques telles que **NumPy**, **SciPy**, **Matplotlib**, **Scikit-learn**, etc, python est le langage le plus utilisé en ML et DL.
- **MatLab** : environnement de calcul numérique multi-paradigme et également langage de programmation de la firme **MathWorks**. Il offre une boîte à outil pour le DL incluant CNN et LSTM.

Étapes d'un processus d'apprentissage profond dans le cadre de la modélisation prédictive

En général, le processus d'apprentissage profond dans le contexte de la modélisation prédictive, consiste en 5 étapes :

1. **Définition et préparation du problème** : consiste à comprendre le problème et son domaine, et à comprendre, collecter et charger tout ce dont on a besoin pour commencer à travailler sur son projet.
2. **Résumé et compréhension des données** : consiste à comprendre les observations du domaine étudié (distribution), et à résumer et visualiser ces observations pour les comprendre plus en détail et être en mesure de voir les tendances. Chaque donnée fournit une petite fenêtre sur un aspect limité de la réalité. Le recueil de toutes ces observations nous donne une image de l'ensemble.

3. **Traitement et préparation des données** : comprend le nettoyage des données en supprimant les doublons, en marquant les valeurs manquantes et même en imputant les valeurs manquantes, la sélection et le raffinement des attributs pertinents, et la transformation des données pour les mettre à l'échelle.
4. **Évaluation des algorithmes** : consiste à analyser les résultats obtenus par les algorithmes utilisés pour l'apprentissage.
5. **Amélioration des résultats** : voir les changements à effectuer pour augmenter la performance du réseau.

Inconvénients de l'apprentissage profond

1. Nécessite un grand volume de données.
2. Modèles boîtes noires (comme les réseaux de neurones artificiels).
3. Temps considérable en phase de développement : il faut trouver le bon réseau de neurones, les bons outils, et le meilleur paramétrage du modèle.
4. Temps d'apprentissage très grand en comparaison avec les autres techniques.

